

SavePoint

Catalogación de videojuegos y comparación reproducible de sistemas de recomendación

Informe de estado para el tribunal — qué se ha construido y qué falta

Trabajo Fin de Grado · ETSII, Universidad de Sevilla

Autor: Felipe Peña Núñez

Fecha del informe: 7 de septiembre de 2026

Demo pública: <https://save-point-orpin.vercel.app>

API: <https://savepoint-api-37nz.onrender.com>

1. Resumen ejecutivo

SavePoint es una aplicación web para catalogar la colección y el *backlog* personal de videojuegos (inspirada en Goodreads y Letterboxd) y, a la vez, una plataforma académica para implementar y comparar recomendadores basados en contenido, colaborativos e híbridos mediante experimentos reproducibles, usuarios sintéticos, resultados explicables y procedencia de datos documentada.

El proyecto se ejecuta por fases, cada una entregada como *slice* vertical mínima con evidencia de tesis contemporánea. **Están cerradas y aceptadas por el autor la Fase 1 (demo pública en tres días) y la Fase 1.1 (catálogo a escala real).** La **Fase 2 está en curso** (ola 4 de 6): corpus gobernado, ratings externos con procedencia, búsqueda tolerante y filtros, protocolo de evaluación congelado y el primer recomendador avanzado. El resto de fases (3–7: comparación completa de recomendadores, colaborativo/híbrido, flujos de colección, descubrimiento público y panel de investigación con endurecimiento) están planificadas pero no iniciadas.

Estado en una línea: aplicación desplegada y reproducible en local, con catálogo real de ~312.000 obras, ratings externos, login, colección completa, interfaz bilingüe y temas claro/oscuro, dos recomendadores base y toda la infraestructura de evaluación congelada; falta cerrar el laboratorio de recomendadores avanzados y su comparación formal.

2. Tecnología utilizada (por capa)

El TFG parte sin *stack* prescrito: toda elección está justificada por idoneidad, reproducibilidad, mantenibilidad y las restricciones de una demostración académica. Se pinta la versión exacta de cada dependencia y se bloquean los *lockfiles*.

2.1. Entorno de desarrollo y reproducibilidad local

Pieza	Elección	Función
Contenedores	Docker Engine + Compose v2	Servicios <code>db</code> , <code>api</code> , <code>web</code> ; Dockerfiles multi-stage, usuario <code>no-root</code> , healthchecks, imágenes fijadas por digest
Entorno Python	<code>uv</code> + <code>pyproject.toml</code> + <code>uv.lock</code>	Python y todas las dependencias transitivas fijadas; grupos <code>app</code> / <code>research</code> / <code>dev</code>
Entorno frontend	<code>pnpm</code> + <code>pnpm-lock.yaml</code> (Corepack)	Instalaciones <code>--frozen-lockfile</code>
Arranque local	<pre>docker compose -f infra/compose.yaml up --build --wait</pre>	Construye las 3 imágenes desde el mismo <code>pyproject/uv.lock</code> y <code>package.json/pnpm-lock</code> que producción; ningún paso de build descarga catálogo ni llama a proveedores externos — el catálogo, las portadas referenciadas y las interacciones de la demo son artefactos versionados en <code>data/</code>
Secuencia de arranque de <code>api</code>	<code>migrate</code> → <code>import_catalogue</code> → <code>bootstrap_demo_account</code> → <code>seed_demo</code> → servidor	Cuatro comandos idempotentes; repetir <code>up</code> no duplica filas

Gate de secretos	<code>scripts/check-secrets.ps1 + manage.py check --deploy</code>	Se autoverifica contra canarios sintéticos y escanea 5 superficies (Git, build estático de Next, capas Docker, logs, <code>.env.example</code>)
------------------	---	--

Entorno probado: Windows 11 (build 26200), Docker Desktop, PostgreSQL 18.6 oficial. Apagar la red externa después de `up --wait` no afecta al recorrido: catálogo, login, valoraciones, copias y perfiles sólo leen PostgreSQL local.

2.2. Backend y base de datos

Tecnología	Versión	Rol y motivo
Python	3.13.x	Un solo lenguaje para Django, preparación de datos, baselines, evaluación y reproducibilidad; línea 3.13 por máxima compatibilidad de <i>wheels</i> científicas
Django	5.2.x LTS	Dominio relacional (catálogo, ediciones, copias, listas, estados, valoraciones, procedencia); auth, ORM, migraciones y admin maduros; soporte LTS hasta abril de 2028
Django REST Framework	3.18.0	API JSON para el cliente web; reglas de negocio en servicios/modelos, no en serializers
PostgreSQL	18.6	Base transaccional y de metadatos de experimentos; JSONB y búsqueda de texto cuando se justifica, sin añadir otro almacén
psycopg	3.x	Driver recomendado por Django

2.3. Frontend

Tecnología	Versión	Rol y motivo
Next.js	16.x (App Router)	SSR y metadatos para perfiles públicos, más interacción cliente para colección y paneles; corre como servidor Node en contenedor
React / React DOM	19.2.x	Componentes accesibles reutilizables (tarjetas, formularios, tablas, gráficas)
TypeScript	6.x, modo estricto	Contrato de API, variantes de inventario, explicaciones de recomendación y esquemas de resultados explícitos
Node.js	24 LTS	Build y runtime frontend
Tailwind CSS	4.x	Tokens de diseño y estilo responsive sin comprometerse con un framework de componentes grande
TanStack Query	5.x	Caché de estado de servidor: mutaciones autenticadas, paginación, invalidación

2.4. Recomendadores y experimentación

Para la Fase 2 se ratificó (2026-09-07) implementar el laboratorio de contenido en **Python puro + biblioteca estándar** (vectores de features, perfil de usuario y coseno escritos a mano; sin NumPy ni scikit-learn todavía), por transparencia y reproducibilidad a la escala actual. El resto del *tooling* científico se incorpora cuando el volumen de interacciones y la pregunta de investigación lo justifiquen (Fase 3 en adelante):

Biblioteca	Versión	Uso previsto
------------	---------	--------------

NumPy	2.5.2	Arrays y trabajo numérico determinista: vectores de features, rankings, métricas, generación sintética con semilla
SciPy	1.18.1	Matrices dispersas para interacciones usuario-ítem (evitar densas que escalan con cada par usuario-juego)
pandas	3.0.x	Ingesta, limpieza y resultados tabulares en las fronteras de datos/experimento
scikit-learn	1.9.0	Features de contenido (TF-IDF/one-hot), similitud coseno, <i>pipelines</i> reproducibles, baselines clásicos y particiones
implicit	0.7.2	ALS/BPR e item-item para retroalimentación implícita (variante experimental aislada)
Optuna	4.x	Búsqueda de hiperparámetros reproducible: <i>sampler</i> con semilla, espacios acotados, <i>trials</i> almacenados, split de validación fijo
MLflow	3.15.2	Seguimiento de experimentos: huella de dataset, commit de Git, semilla, split, parámetros, métricas, tiempos, lock de entorno y artefactos
joblib	con scikit-learn	Evaluación paralela conservadora y serialización de artefactos

2.5. Pruebas y calidad

Herramienta	Uso
pytest 9 + pytest-django	Tests unitarios/integración contra PostgreSQL real (no SQLite): servicios de dominio, permisos, import/export, fórmulas de métricas, generación y split deterministas
Hypothesis	Pruebas basadas en propiedades: invariantes de ranking, cotas de métricas, <i>round-trips</i> de import
Vitest + Testing Library	Componentes y funciones puras del frontend; consultas por rol/nombre accesible
Playwright 1.62	E2E de navegador en flujos críticos: login, edición de <i>backlog</i> , inventario de copias, perfil público, recomendación
axe-core (integración Playwright)	Chequeos de accesibilidad automatizados sobre páginas clave
Ruff / mypy + django-stubs / ESLint	Lint y análisis estático de Python y TypeScript

2.6. Producción en web y despliegue

Topología sin tarjeta de crédito, con paridad estricta con local (mismo commit y *lockfiles* construyen local y desplegado):

Componente	Plataforma	URL
Aplicación web (Next.js)	Vercel Hobby	https://save-point-orpin.vercel.app
API (Django + DRF)	Render Free (Docker)	https://savepoint-api-37nz.onrender.com
Base de datos	Neon Free (PostgreSQL gestionado)	—

CI	GitHub Actions	Tests backend contra PostgreSQL, tests frontend, generación/diff de esquema, builds de imagen, humo Playwright, integridad de lockfiles
----	----------------	---

La UI habla con la API sólo a través de un proxy *same-origin* de Next.js (`next.config.ts`); el navegador nunca hace una petición cross-origin directa. El entrenamiento/recálculo de recomendaciones se mantiene como trabajo explícito fuera de las peticiones HTTP.

2.7. APIs externas y su procedencia

Fuente	De dónde sale	Uso en SavePoint	Marco legal / términos
IGDB v4	Base de datos de videojuegos de Twitch/Amazon ; API autenticada vía credenciales de Twitch	Fuente del catálogo a escala real. Sonda autenticada (2026-09-05): ~374.555 juegos totales, ~312.418 primarios (<code>game_type = 0</code>); sin cuota mensual; 4 req/s + 8 concurrentes. Frontera de import: <code>game_type = 0</code> . Portadas <i>hotlinked</i> a <code>images.igdb.com</code> (no se replican), con el placeholder de primera parte como respaldo	Twitch DSA (caché 24 h, sin redistribución) reconciliado con la FAQ de la propia API de IGDB (permite almacenar y servir, y conservar tras la terminación), tratada como la "autorización escrita" que contempla la DSA. Posición razonada registrada literalmente para la memoria — no asesoramiento legal. ADR-006
RAWG API	<code>rawg.io</code> , agregador de datos de videojuegos	Enriquecimiento con ratings de usuario como segunda fuente, con procedencia. Pasada $N=10.000$ sobre el <i>top</i> por <code>rating_count</code> (2026-09-07): 8.575 obras emparejadas, 8.575 <i>snapshots</i> <code>CorpusRatingSnapshot(source="rawg")</code> , 0 cobertura nueva (ese corte ya tenía rating de IGDB, así que aquí RAWG es contraste de fuente, no relleno)	Términos Free: uso no comercial, atribución + <i>backlink</i> , 20.000 req/mes, sin redistribución. ADR-008. La pasada se mantuvo muy por debajo de cuota, sin 429
Wikidata	Consulta SPARQL (<code>wdt:P31 wd:Q7889</code>) filtrada por clase, más subconsultas de plataforma resueltas dinámicamente	Corpus <i>lawful</i> offline de la Fase 1 (demo); se conserva como corpus offline reproducible sin dependencia de red	Licencia CC0 de Wikidata; procedencia y fecha de recuperación registradas

Regla transversal: durante la **evaluación** nunca se llama a una API en vivo — se usa un *snapshot* inmutable con procedencia y *checksum*. La API en vivo es sólo para enriquecer la presentación del catálogo.

3. Lo que ya está implementado HECHO

3.1. Catálogo funcional a escala real

- **Import IGDB completo:** 312.560 obras en base de datos (cursor 416.648), verificado por consulta directa.

- **Corpus gobernado:** 193.885 obras tras aplicar una *allowlist* documentada de plataformas y calidad (Steam incluido) y reglas de exclusión de títulos basura. El comando `govern_corpus` produce un *checksum* fijo, un diccionario de datos y un informe de calidad/campos ausentes (DATA-03), de forma independiente del enriquecimiento mutable.
- **Búsqueda tolerante:** *backfill* de `GameAlias` y coincidencia aproximada de título sobre el corpus real (plan 02-03).
- **Filtros multiselección** de varios géneros y varias plataformas a la vez, a nivel de backend; los filtros intersecan y `sort` es un *allowlist* fijo, nunca interpolado (plan 02-03).
- Serializer de catálogo y endpoints para el pase de UI product-grade (`display_rating` relabelado, obras propias con DLC), plan 02-05.

3.2. Ratings externos con procedencia e aislamiento de experimentos

- Campos de rating de usuario de IGDB importados: **27.014 obras con rating** (13,933 % de densidad — la densidad real de IGDB, no un hueco del import).
- `CorpusRatingSnapshot` **immutable** por `corpus_version`: cada valor guarda su fuente y fecha de recuperación, y un experimento cerrado nunca puede alterarse por datos posteriores del proveedor (DATA-06). `GameWork.rating` no se muta.
- RAWG añadida como segunda fuente con su propia procedencia (ADR-008).

3.3. Recomendaciones base

- **Popularidad (REC-02)** — **HECHO** baseline no personalizado desde los datos de la demo (Fase 1).
- **Gusto por género `genre-taste-v1` (REC-10)** — **HECHO** heurística determinista en tiempo de petición sobre la biblioteca del propio usuario: pondera géneros por estado + rating, ordena obras no vistas no-DLC por rating de catálogo dentro de los géneros compartidos, con *fingerprint* SHA-256 de entrada y salida y un estado explícito de "historial insuficiente" que nunca cae al baseline público. Endpoint `GET /api/recommendations/genre-taste/`. ADR-007. (Es una *feature* de producto, deliberadamente más simple que el recomendador de contenido de la tesis; no es la contribución algorítmica.)
- **Baseline aleatorio `random-v1` (REC-01)** — **EN CÓDIGO** sorteo uniforme con semilla sobre el corpus gobernado menos la biblioteca del usuario; suelo de comparación (plan 02-10, cierre formal pendiente).

3.4. Cuentas y login

- Registro con lista concreta de errores de contraseña, login insensible a mayúsculas, `GET /api/accounts/me/`.
- Cuentas demo controladas y rotables (`bootstrap_demo_account`).
- En la demo toda cuenta es pública por diseño; el serializer de perfil público es una *allowlist* escrita a mano (nunca introspección automática), verificada con escaneo recursivo de *denylist* y tests XSS hostiles.

3.5. Interfaz bilingüe español / inglés

- i18n con diccionarios `es.ts` / `en.ts`, rutas `/es` y `/en`, y un test de paridad de claves que impide que una traducción quede coja.

3.6. Temas claro y oscuro

- `ThemeToggle` en el *AppShell*, sobre tokens de diseño en `globals.css`.

3.7. Colección e inventario

- *Backlog* con estados (pendiente, jugando, completado, abandonado), valoración por medios pasos, e **inventario de copias físicas y digitales** con plataforma, edición, datos de compra y estado de conservación.
- Perfil público que expone sólo campos en *allowlist*; los detalles privados de propiedad no aparecen.
- El E2E prueba la persistencia de valoración y de múltiples copias tras recargar, no sólo el campo de estado.

3.8. Accesibilidad y diseño responsive

- Navegable por teclado y responsive a anchos de escritorio y móvil; chequeos axe-core automáticos sobre páginas clave.
- *Limitaciones abiertas declaradas* (de la Fase 1, no ocultas): el *reflow* a 400 % de zoom y un pase con lector de pantalla siguen sin verificarse manualmente.

3.9. Infraestructura de investigación (congelada)

- **Protocolo de evaluación congelado** (`docs/methodology/protocol.json`): 12 claves + `simulation: true`, rejilla de tuning de 18 configuraciones (tope duro 24), documento `evaluation-protocol.md` con sección "Amenazas a la validez" (DOC-04). La app `evaluation` está en `INSTALLED_APPS` con `protocol.py`, `metrics.py` (precision/recall/nDCG/MAP a mano) y `splits.py` (leave-one-out por usuario + partición train/val/test 120/40/40). `pytest apps/api`: 280/280 en verde.
- **Usuarios sintéticos reproducibles por semilla (EVAL-09/EVAL-10)**: 200 usuarios (25 de arranque en frío) generados desde arquetipos parametrizados contrastados, con historias idénticas por semilla, aislados del baseline de popularidad, con informe de validación. 17 tests específicos + 352 del backend en verde.
- Modelo de features de contenido y caché `WorkFeatureVector` (plan 02-10): vector disperso por obra (género + plataforma, peso $1/\sqrt{k}$ por faceta, versión `fs-v1`), perfil de usuario como media ponderada por actividad de vectores L2-normalizados, y coseno sobre *dicts* dispersos con `math.fsum`.

4. Lo que falta a corto plazo

4.1. Pestaña "Novedades" PENDIENTE — e investigar el cómo

Estado actual

Ya existe el **endpoint de backend** `GET /api/catalogue/new-releases/` (decisión D-24): devuelve hasta 20 obras *gobernadas* cuya `first_release_date` cae en los últimos ~6 meses, ordenadas por fecha descendente con `canonical_slug` como desempate determinista; tiene su propio *throttle* (120/min) y una ventana vacía devuelve `[]` con 200 (nunca un error). **No hay todavía una pestaña ni una página dedicada** en la navegación (hoy: Inicio, Catálogo, Colección, Recomendaciones, Fuentes). El plan 02-06 la incorpora como estante en la portada ("Novedades" + "Para tus juegos").

Cómo hacerlo y qué hay que investigar

- **UI**: añadir la entrada de navegación y una página `/es/novedades` con paginación (el endpoint ya acepta `page`), reutilizando `GameCardSerializer`.
- **Frescura de los datos** (el punto de investigación real): el catálogo actual es un *import congelado*. Unas "novedades" reales exigen una **pasada de re-import incremental periódica desde IGDB** filtrando por `first_release_date` / `updated_at`. Hay que definir: la cadencia (p. ej. semanal), que sea un *job* explícito fuera de las peticiones HTTP, y cómo se registra su procedencia sin romper el *snapshot* de investigación.
- **Criterio de "novedad"**: ventana móvil de fecha de lanzamiento frente a fecha de alta en el corpus; decidir si se incluyen lanzamientos futuros (fechas por venir) o sólo títulos ya publicados.
- **Gobernanza y aislamiento**: el estante ya respeta la *allowlist* del corpus gobernado (usa `governed_works()`) y es sólo lectura de catálogo, por lo que no contamina datos de investigación — conviene dejarlo verificado por test.

4.2. Más ratings PENDIENTE

- **RAWG pasada 2** (ratificada por el autor, 2026-09-07): juegos con `rating IS NULL` y `first_release_date` entre 2022 y 2026, ordenados por popularidad (`total_rating_count` → `rating_count` → recientes), `--limit 20000` hasta agotar cuota. Requiere *flags* nuevos en `enrich_rawg_ratings` (`--only-unrated`, `--release-year-min/max`, `--order-by popularity`), tests y una enmienda a ADR-008. Motivo del autor: *máxima cantidad de datos ahora*.
- **Otras APIs al final** para rellenar el resto de cobertura (p. ej. Steam / Metacritic / OpenCritic u otras), sujeto a sus términos, con la misma disciplina de *snapshot* inmutable y procedencia.

4.3. Revisar el corpus — qué entra en la página PARCIAL

- Verificar el corpus gobernado por su *checksum* fijo, diccionario de datos e informe de calidad (DATA-03, hecho) y **ratificar la allowlist de plataformas y calidad** y las reglas de exclusión de basura.
- **Reconciliación determinista de identificadores** y reglas de conflicto entre fuentes (DATA-07) — pendiente.
- **Defecto pre-existente registrado y diferido**: 22 ids de RAWG (0,26 %) reconcilian a 2 obras gobernadas cada uno; los ratings son correctos (cada obra conserva su *snapshot* inmutable), sólo la procedencia RAWG de esas filas es imprecisa; la corrección es un cambio de esquema de procedencia.
- **Pase de UI product-grade** sobre catálogo, filtros y ficha de juego (densidad, tipografía, imágenes) — planes 02-04 y 02-06.

5. Algoritmos complejos: ¿definidos? ¿desarrollados? ¿cuáles? ¿cómo?

5.1. ¿Están definidos?

Sí, en detalle. El protocolo de comparación se ha *congelado* (decisión de un solo sentido) *antes* de comparar cualquier recomendador avanzado. Está fijado en `protocol.json` y en `docs/methodology/evaluation-protocol.md`, con las decisiones D-10...D-22 razonadas y una sección de "Amenazas a la validez":

Parámetro	Valor congelado
-----------	-----------------

Relevancia	obra <code>completed</code> y <code>rating_half_steps</code> ≥ 7 ($\approx 3,5/5$)
K	{5, 10, 20}; métrica titular nDCG@10
Split	leave-one-out por usuario, semilla 20260907; el <i>held-out</i> vuelve al conjunto de candidatos
Conjunto de candidatos	corpus gobernado – biblioteca del usuario + <i>held-out</i>
Exclusiones	cualquier <code>LibraryEntry</code> del usuario
Métricas	<code>precision@k</code> , <code>recall@k</code> , <code>ndcg@k</code> , <code>map@k</code>
Tuning	rejilla de 18 configuraciones (3 modos de combinación $\times$ 3 conjuntos de features $\times$ parámetros); selección por nDCG@10 en validación ; 1 sola ejecución de test
Partición de usuarios	train 120 / validación 40 / test 40, semilla 20260908

El código del protocolo se niega a ejecutarse sobre una rejilla > 24 o un split de test ya consumido.

5.2. ¿Están desarrollados?

Parcialmente.

- **EN CÓDIGO** baseline aleatorio (`rank_random_v1`); vectores de features de contenido; perfil de usuario ponderado por actividad; similitud coseno dispersa (`stdlib`); caché `WorkFeatureVector`; estadística de rating por género leída del *snapshot* inmutable.
- **PENDIENTE — plan 02-11 (ola 4)** cerrar las tres variantes con nombre y versión del recomendador de contenido, el término de rating del candidato, la explicación determinista, el arranque en frío, las exclusiones y el endpoint `ContentRecsView`.
- **PENDIENTE — plan 02-13 (ola 5)** la primera comparación completa con `run_evaluation` (EVAL-01) más los controles metodológicos (AGENT-04) y la reconciliación (DATA-07).

5.3. ¿Cuáles vamos a implementar?

Baselines (suelos de comparación)

- **Aleatorio** (REC-01) y **popularidad** (REC-02, hecho).

Basado en contenido (REC-03) — primera contribución algorítmica del TFG

Se construye como un "laboratorio" de **tres variantes con nombre y versión**, todas comparables bajo el protocolo congelado:

Variante	Combinación de puntuación
<code>content-cbf-weighted-v1</code>	$w1 \cdot \text{cos} + w2 \cdot \text{término_rating} (+ w3 \cdot \text{rating_propio})$
<code>content-cbf-multiplicative-v1</code>	$\text{cos} \cdot \text{término_rating}$
<code>content-cbf-twostage-v1</code>	bandas de similitud, desempate por rating dentro de cada banda

- **Término de rating del candidato** (D-13): mezcla por confianza (según `rating_count`) entre el rating propio del candidato (del `CorpusRatingSnapshot`) y el perfil de rating por género (media del *snapshot* por género), normalizado a [0,1]; si el candidato no tiene rating propio se usa la mediana de sus

géneros y se marca `rating_term_is_fallback`. Se lee **siempre** del *snapshot*, nunca `GameWork.total_rating`.

- **Explicación** (REC-08): tabla determinista de contribución por género (participación de cada dimensión en el numerador del coseno, ordenada) + valor del término de rating + variante que la produjo. Sin prosa generada.
- **Arranque en frío** (REC-06): < 3 entradas de biblioteca con género → `insufficient_history` y una vista de *fallback* por géneros populares; nunca una lista vacía.
- **Exclusiones** (REC-07): toda obra con cualquier `LibraryEntry` del usuario.
- **Versiónado** (REC-09): el DTO lleva `feature_set_version`, `corpus_version` y `snapshot_sha256` de cada resultado publicado.

Fase 3 — comparación explicable y baselines

Correr el *harness* congelado sobre los baselines y todas las variantes de contenido con enrutado de arranque en frío, exclusiones y explicaciones deterministas, la suite completa de métricas (precisión/ranking + **cobertura, diversidad y novedad**), informe por cohorte (incluidos usuarios de cero historial e historial escaso), múltiples semillas, estimaciones de incertidumbre y tests estadísticos justificados, y artefactos inmutables que permiten recalcular cada resultado sin re-ejecutar ningún modelo.

Fase 4 — colaborativo e híbrido

Al menos un método de **filtrado colaborativo** (REC-04) y un **híbrido** documentado (REC-05) bajo el mismo protocolo ya congelado. El híbrido combina evidencia de contenido y colaborativa y transiciona de forma predecible para usuarios fríos y escasos. Candidatos: k-NN item-item / user-user y factorización matricial (o `implicit` ALS/BPR como variante experimental aislada). No se usan recomendadores neuronales por defecto: con datos sintéticos/controlados no sostendrían mejoras defendibles y aumentarían las amenazas a la validez.

5.4. ¿Cómo lo vamos a hacer? (metodología)

- **Evaluación offline congelada y consciente de la simulación**: cada número es evidencia sobre arquetipos sintéticos parametrizados, no sobre usuarios reales de SavePoint (EVAL-10).
- **Usuarios sintéticos reproducibles por semilla** (200, 25 de arranque en frío) desde arquetipos contrastados, con historias idénticas por semilla e informe de validación.
- **Métricas implementadas y testeadas a mano**: el protocolo no depende de una librería externa.
- **Rejilla de tuning acotada** (18 configuraciones, tope duro 24), selección en validación, una sola pasada de test.
- **MLflow** para registrar huella de dataset, commit, semilla, split, parámetros, métricas, tiempos, lock de entorno y artefactos; artefactos inmutables recalculables.
- **Stack**: Python puro en la Fase 2 (decisión ratificada 2026-09-07); NumPy / SciPy sparse / scikit-learn / implicit / Optuna se incorporan cuando el volumen y la pregunta de investigación lo justifiquen (Fase 3+).
- **Panel de investigación accesible** con tablas y exportaciones *thesis-ready* en la Fase 7.

6. Riesgos y limitaciones conocidas

- Densidad de rating baja (13,933 %); la pasada 1 de RAWG no añadió cobertura neta. Mitigación: pasada 2 + otras APIs al final.

- Los resultados de recomendación serán evidencia de *simulación*, no de usuarios reales — así se declara en el protocolo y en las conclusiones.
- Defecto de procedencia de 22 filas de RAWG (0,26 %) diferido a un cambio de esquema.
- *Reflow* a 400 % de zoom y pase con lector de pantalla pendientes de verificación manual (heredado de la Fase 1).
- Las Fases 3 y 4 todavía no tienen planes detallados (marcadas TBD en el *roadmap*).
- La memoria LaTeX se compila en Overleaf (no hay TeX instalado en local); el ZIP definitivo se regenerará tras cerrar la Fase 2 con sus algoritmos y decisiones incorporados.

7. Próximos pasos concretos

1. **Fase 2, ola 4:** plan 02-04 (UI de filtros multiselección y densidad product-grade del catálogo) y plan 02-11 (variantes del recomendador de contenido, explicación, arranque en frío, endpoint).
2. **RAWG pasada 2** (juegos sin rating, 2022–2026, por popularidad).
3. **Fase 2, ola 5:** plan 02-06 (ficha de juego product-grade + estantes "Novedades" y "Para tus juegos") y plan 02-13 (primera comparación completa `run_evaluation`).
4. **Fase 2, ola 6:** plan 02-12 (página de recomendaciones con tres secciones etiquetadas que coexisten: contenido / género / DLC).
5. **Planificar las Fases 3 y 4** (comparación completa de recomendadores; colaborativo e híbrido).

Progreso global: 2 de 8 fases completas (25 %). Fase 2: 34 de 39 planes de la fase ejecutados; olas 1–3 cerradas, ola 4 desbloqueada.